

Relatório do Projeto URL Shortener

Felipe de Souza Goulart (19200417)

Henrique Afonso Alves de Lima (23150563)

1. Decisões de Implementação

1.1 Arquitetura

O projeto adota uma arquitetura baseada no modelo cliente-servidor, organizada em três componentes principais: um servidor REST em C++, um cliente em TypeScript e um proxy em C++. Nessa organização, o servidor oferece os serviços de encurtamento, resolução e remoção de URLs. O cliente consome esses serviços remotamente e o proxy foi concebido como um componente intermediário com uso de cache.

Essa separação em componentes está de acordo com o modelo de sistemas distribuídos apresentado nas aulas, no qual processos clientes enviam requisições a processos servidores que oferecem serviços específicos. A presença de um proxy também se relaciona ao conteúdo de arquiteturas distribuídas, em que componentes intermediários podem implementar funções como cache, intermediação entre cliente e servidor e melhoria de desempenho.

Além disso, o projeto utiliza comunicação remota por requisição e resposta sobre HTTP, o que o aproxima do modelo de serviços web discutido na disciplina. Como os componentes são implementados em linguagens diferentes, C++ e TypeScript, o sistema também apresenta um grau de heterogeneidade, resolvida pelo uso de protocolos e formatos comuns de comunicação.

1.2 Escolha de Tecnologias

No **servidor**, foi utilizada a linguagem C++ com padrão C++20, em conjunto com as bibliotecas `cpp-httplib` para implementação do servidor HTTP e `nlohmann/json` para manipulação de mensagens em JSON. Essa escolha permitiu implementar diretamente os serviços REST e controlar a lógica de armazenamento e processamento das requisições.

No **cliente**, foi utilizado TypeScript com TCP sockets à API. Essa escolha favorece simplicidade de uso, manutenção do código e separação entre a lógica de consumo do serviço e a implementação interna do servidor.

No **proxy**, também foi adotado C++, mantendo consistência com o servidor. Os arquivos disponíveis mostram a presença de um cliente REST e de uma estrutura de cache LRU, indicando que o componente foi projetado para atuar como intermediário entre cliente e servidor, com foco em reaproveitamento de respostas recentes.

2. Estrutura do Projeto

A organização do projeto pode ser representada da seguinte forma:

```
url-shortener/
├── client_py/
│   ├── __init__.py # módulo Python
│   ├── example.py # arquivo de exemplo de uso
│   └── url_shortener_client.py # cliente Python principal
├── client_ts/
│   ├── src/
│   │   ├── UrlShortenerClient.ts # cliente TypeScript principal
│   │   ├── index.ts # ponto de entrada da biblioteca
│   │   └── test.ts # arquivo de testes
│   ├── package.json # configuração do projeto Node.js
│   └── tsconfig.json # configuração do TypeScript
├── proxy/
│   ├── cache/
│   │   └── LRUCache.hpp # implementação de cache LRU
│   ├── http/
│   │   ├── RestClient.cpp # cliente HTTP
│   │   └── RestClient.hpp # header do cliente HTTP
│   ├── patterns/
│   │   ├── CircuitBreaker.cpp # implementação do circuit breaker
│   │   └── CircuitBreaker.hpp # header do circuit breaker
│   ├── protocol/
│   │   ├── MessageParser.cpp # parser de mensagens
│   │   └── MessageParser.hpp # header do parser
│   ├── CMakeLists.txt # configuração de build do proxy
│   └── main.cpp # ponto de entrada do proxy
├── server/
│   ├── CMakeLists.txt # configuração de build do servidor
│   ├── UrlStore.cpp # armazenamento de URLs
│   ├── UrlStore.hpp # header do armazenamento
│   └── main.cpp # ponto de entrada do servidor
├── .gitignore # regras de ignore do Git
└── Makefile # automação de build
```

3. Compilação e Execução dos Testes

Pré-requisitos

- Compilador C++ com suporte a C++20
- CMake 3.16 ou superior
- Node.js 18 ou superior

```
make all
make run-server
make run-proxy
#Para rodar o client python
make test-py
#Para rodar o client typescript
cd client_ts
npm install
npm start
...
```

4. Proxy

Funcionamento

O proxy atua como interceptador. Ele recebe as mensagens via socket, interpreta o comando, aplica as estratégias de cache e tolerância a falhas, e então encaminha a requisição para o servidor REST via HTTP.

Cache

O proxy utiliza o padrão Cache-Aside para resoluções de URLs. Quando um cliente solicita **RESOLVE <código>**, o proxy primeiro consulta seu cache local. Se a entrada existir, ocorre um **CACHE HIT**, e a resposta é devolvida diretamente ao cliente, sem chamada ao servidor REST. Se a entrada não existir, ocorre um **CACHE MISS**, o proxy consulta o servidor REST, armazena a resposta no cache e retorna a URL original ao cliente.

A cache foi implementada com política LRU, com capacidade atual de 5 entradas, para facilitar a visualização nos testes. Quando uma sexta entrada é inserida, a entrada menos recentemente usada é removida. Quando uma URL é removida com **REMOVE <código>**, o proxy também invalida a entrada correspondente no cache, evitando retornar dados obsoletos.

Circuit Breaker

Como segundo padrão de projeto, foi escolhido o Circuit Breaker, com o objetivo de melhorar a tolerância a falhas. O proxy monitora falhas ao chamar o servidor REST. Após 3 falhas consecutivas, o circuito muda para o estado OPEN, e novas requisições deixam de ser encaminhadas ao servidor, retornando diretamente erro ao cliente. Isso evita insistir em chamadas para um serviço indisponível.

Após 10 segundos, o circuito passa para **HALF_OPEN**, permitindo algumas tentativas de teste. Se 2 chamadas forem bem-sucedidas, o circuito volta para **CLOSED**. Se houver falha nesse estado, ele retorna para **OPEN**.

O cliente utiliza comunicação TCP direta com o proxy através dos seguintes comandos:

Formato	Descrição
ENCURTA <url>	Encurta uma URL.
RESOLVE <code>	Retorna a URL original associada ao código.
REMOVE <code>	Remove uma URL armazenada.

Exemplos de comunicação:

ENCURTA http://exemplo.com → OK ABC123 http://127.0.0.1:8080/ABC123

RESOLVE ABC123 → OK http://exemplo.com

REMOVE ABC123 → OK

5. API REST do lado Servidor

A API expõe os seguintes métodos e endpoints:

Método	Endpoint	Descrição
POST	/urls	Encurta uma URL.
GET	/urls/{code}	Retorna a URL original associada ao código.
DELETE	/urls/{code}	Remove uma URL armazenada.

POST /shorten

Request

```
{ "url": "https://example.com" }
```

Response (201)

```
{ "code": "abc123" }
```

GET /resolve/{code}

Response (200)

```
{ "url": "https://example.com" }
```

DELETE /remove/{code}

Response (200)

```
Status OK
```

6. Fluxo de Encurtamento de URL

O encurtamento é a operação central do sistema. Quando o cliente envia um `ENCURTA <url>` para o proxy na porta 9000, o servidor processa a requisição enviando a mensagem `ENCURTA http://exemplo.com.br` e verifica o cache LRU. Se não encontrar na cache, ele faz uma requisição HTTP para o servidor. O proxy envia `POST /shorten` com JSON para o servidor, que repassa para `UrlStore::shorten()`.

Dentro do `UrlStore`, o método gera um código aleatório de 6 caracteres alfanuméricos e verifica se ele já está em uso. Se houver colisão, o processo se repete até encontrar um código livre. O par `código -> URL` é então inserido em uma estrutura `std::unordered_map` interna, protegida por `mutex` para suportar acessos concorrentes.

O servidor responde com status 201 e o código gerado.

6.1 Resolução de URL

Para recuperar a URL original, o proxy envia um `GET /resolve/{code}`. O servidor usa uma expressão regular para extrair o código do caminho e o repassa para `UrlStore::resolve()`, que faz a busca no mapa. Se o código existir, retorna a URL com status 200; caso contrário, retorna 404.

6.2 Remoção de URL

O `DELETE /remove/{code}` segue a mesma lógica de extração de código via expressão regular. `UrlStore::remove()` tenta apagar a entrada do mapa; se ela existir, a operação retorna 200; se não existir, retorna 404.

As três operações, `shorten`, `resolve` e `remove`, adquirem o `mutex` antes de acessar o mapa interno, o que garante exclusão mútua no acesso ao armazenamento local.

8. Conclusão

Este relatório apresentou a implementação atual do projeto URL Shortener, destacando sua organização em componentes, o funcionamento da API REST e a relação da solução com conceitos de computação distribuída, como arquitetura cliente-servidor, serviços web, proxy, cache, concorrência e escalabilidade.

Embora o sistema ainda tenha limitações importantes, especialmente quanto à persistência, sua estrutura atual já constitui uma base adequada para evolução em direção a uma solução distribuída mais completa.